EXAM CHEAT SHEET

Sorting Algorithms

Binary Search, Bubble, Selection, Insertion, Merge, Quick, Counting, and Radix Sort — with step-by-step visual diagrams, C++ code, complexity analysis, and exam questions.

Algorithms • C++ • Complexity • Visual Traces

SECTION 1

Quick Reference — All Algorithms at a Glance

Comparison sorts are bounded below by Ω(n log n). Non-comparison sorts (Counting, Radix) break this barrier by exploiting key structure such as integer range or digit position. This table summarizes every algorithm covered in this cheat sheet, including their type, time complexity across all three cases, space requirements, and stability. Use this as your first stop when revising.

Algorithm	Type	Best	Average	Worst	Space	Stable
Binary Search	Search (D&C)	Θ(1)	Θ(lg n)	Θ(lg n)	O(1) / O(lg n)	n/a
Bubble Sort	Comparison	O(n)	O(n²)	O(n²)	O(1)	Yes
Selection Sort	Comparison	O(n²)	O(n²)	O(n²)	O(1)	No
Insertion Sort	Comparison	O(n)	O(n²)	O(n²)	O(1)	Yes
Merge Sort	Comparison (D&C)	O(n lg n)	O(n lg n)	O(n lg n)	O(n)	Yes
Quick Sort	Comparison (D&C)	O(n lg n)	O(n lg n)	O(n²)	O(lg n)	No
Counting Sort	Non-comparison	O(n+k)	O(n+k)	O(n+k)	O(n+k)	Yes
Radix Sort	Non-comparison	O(d(n+b))	O(d(n+b))	O(d(n+b))	O(n+b)	Yes

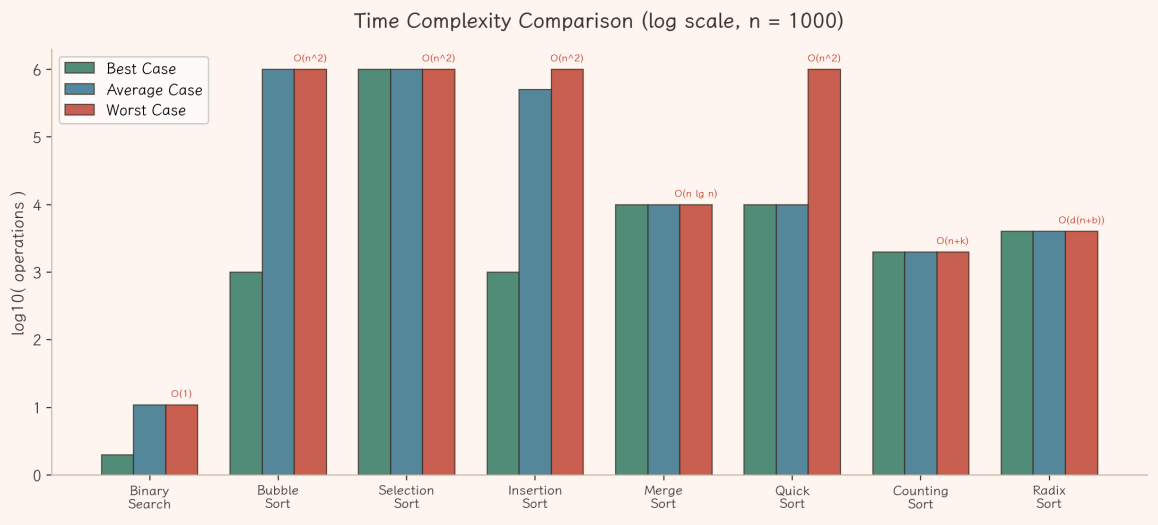


Figure 1: Time complexity comparison across all algorithms (log scale, n = 1000)

1.1 Decision Tree Lower Bound

Theorem: Any comparison sort requires Ω(n log n) comparisons in the worst case. A comparison sort on n elements produces a binary decision tree with n! leaves (one per permutation). If the tree height is h, then $2^h \geq n!$, so $h \geq \lg(n!) = \Omega(n \log n)$. This means Merge Sort and Heap Sort are asymptotically optimal comparison sorts, and Quick Sort is optimal on average. The escape route from this bound is to use information other than comparisons, for example using key values as array indices in Counting Sort.

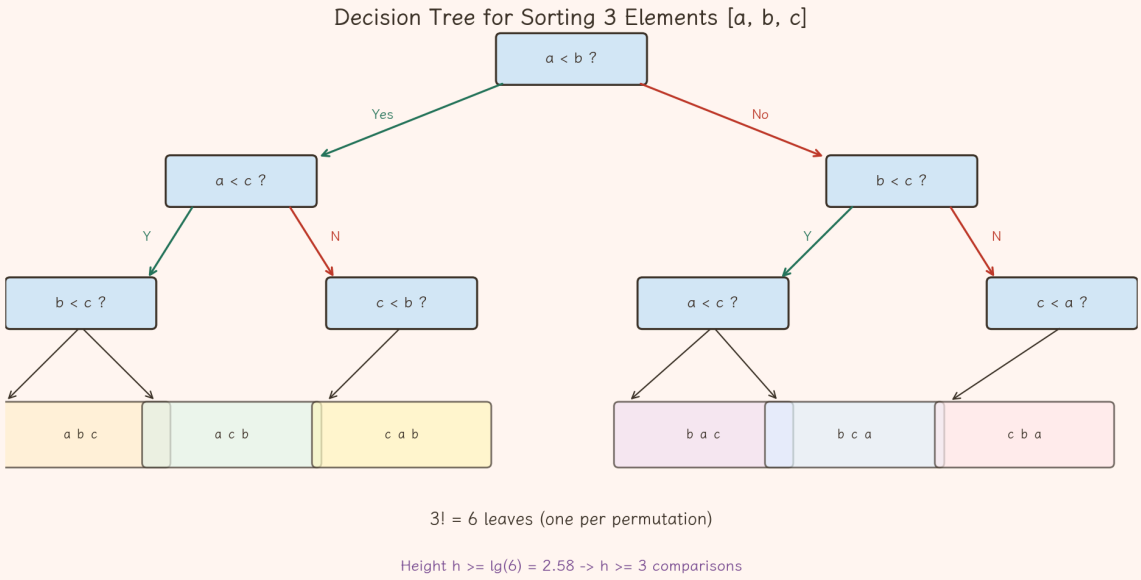


Figure 2: Decision tree for sorting 3 elements — 3! = 6 leaves, height ≥ 3

1.2 Divide & Conquer Template

Binary Search, Merge Sort, and Quick Sort all follow the Divide & Conquer paradigm. First, **Divide** the problem into smaller sub-problems of the same type. Then **Conquer** the sub-problems recursively, with a base case when the problem is small enough to solve directly. Finally, **Combine** the sub-problem solutions into a solution for the original problem — this step may be empty, as in Quick Sort where all work happens during partitioning. The generic recurrence is $T(n) = a \cdot T(n/b) + f(n)$; for Merge Sort, a=2, b=2, $f(n)=\Theta(n)$, yielding $T(n) = \Theta(n \log n)$.

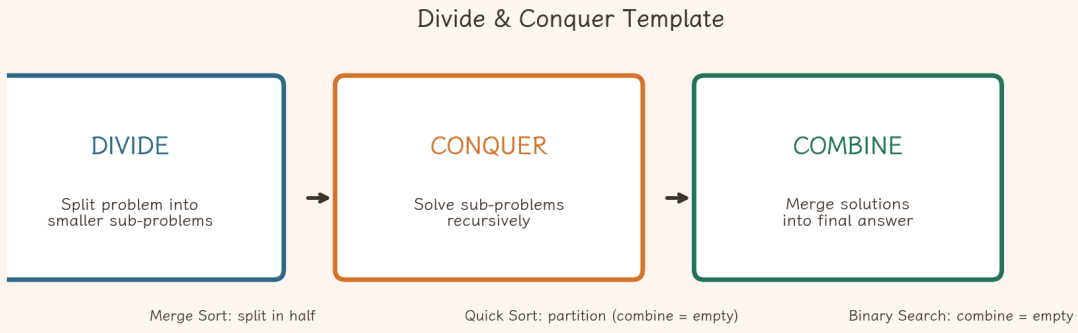


Figure 3: Divide & Conquer template used by Binary Search, Merge Sort, and Quick Sort

SECTION 2

Binary Search

2.1 Basic Idea

Binary search finds a key in a sorted array by repeatedly halving the search interval. It is the textbook example of Divide & Conquer where the combine step is empty. The algorithm requires the input array to be sorted beforehand; otherwise its behavior is undefined. Time complexity is Θ(lg n) since each comparison discards half the remaining elements; space is O(1) for the iterative version or O(lg n) for the recursive version due to the call stack. The key insight is that only one sub-problem is explored per recursive call, giving the recurrence T(n) = T(n/2) + Θ(1) = Θ(lg n).

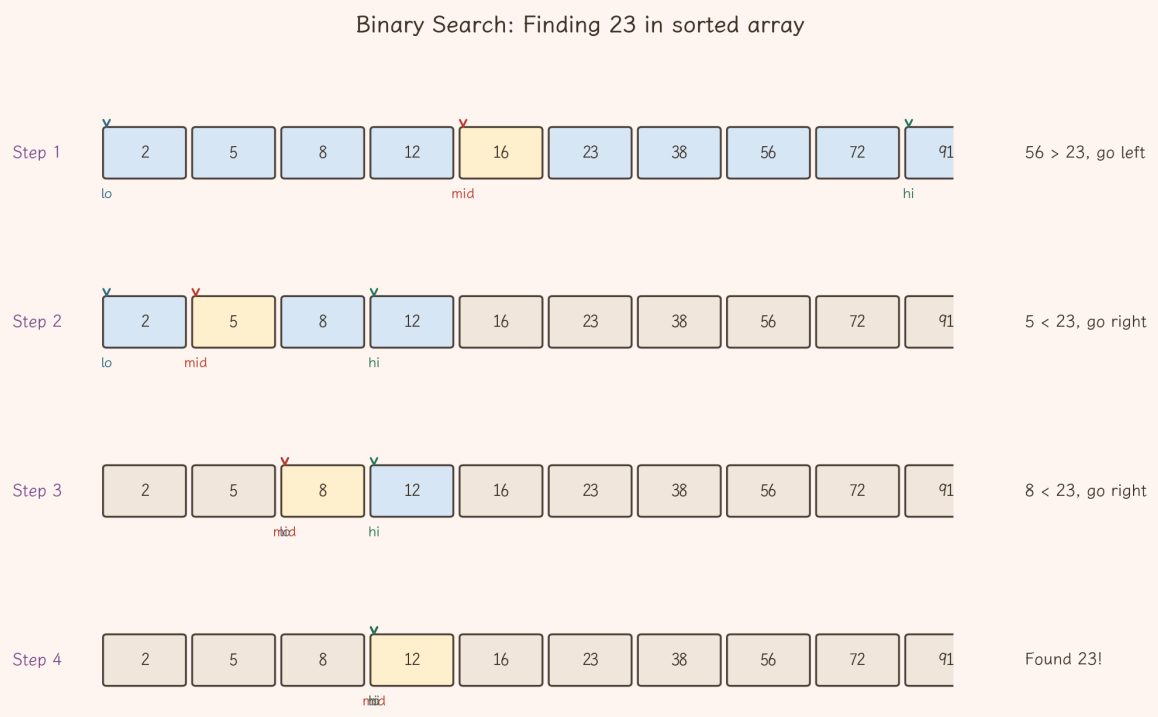


Figure 4: Binary search halving steps — finding 23 in a sorted array

2.2 Algorithm (Recursive)

Objective: find index of x in sorted A[first..last]. **Input:** sorted A, indices first ⩽ last, key x. **Output:** index of x or -1. The algorithm computes mid = first + (last - first) / 2 to avoid integer overflow, compares A[mid] with x, and recursively searches the left or right half accordingly. The initial call is BinarySearch(A, 0, n-1, key).

```
BinarySearch(A, first, last, x)
1. if last >= first then
2.   mid = first + (last - first) / 2  // avoids overflow
3.   if A[mid] == x return mid
4.   else if A[mid] > x return BinarySearch(A, first, mid-1, x)
```

```
5.   else return BinarySearch(A, mid+1, last, x)
6. return -1  // not found
```

2.3 C++ Implementation

```cpp
// Iterative (preferred: O(1) space)
int binarySearch(const vector<int>& A, int x) {
  int lo = 0, hi = (int)A.size() - 1;
  while (lo <= hi) {
    int mid = lo + (hi - lo) / 2; // overflow-safe
    if (A[mid] == x) return mid;
    else if (A[mid] > x) hi = mid - 1;
    else lo = mid + 1;
  }
  return -1;
}
```

Common Mistakes

> Using **mid = (lo+hi)/2** — integer overflow for large arrays. Use lo + (hi-lo)/2.
>
> Wrong loop condition (lo < hi vs lo ≤ hi) — misses single-element range.
>
> Off-by-one in hi = mid-1 / lo = mid+1 — using mid directly causes infinite loop.
>
> Forgetting the array must be sorted first.

SECTION 3

Bubble Sort

3.1 Basic Idea

Bubble Sort repeatedly compares adjacent pairs and swaps them if they are out of order. After pass i, the i-th largest element "bubbles" to its final position at the end of the array. The algorithm is stable and in-place, making it the simplest to understand but the slowest in practice. Without optimization, the time complexity is $O(n^2)$ in all cases, but adding an early-stop flag (a swapped boolean that breaks if no swaps occur in a full pass) improves the best case to $O(n)$ for already-sorted input. The comparison count follows $C(n) = (n-1) + (n-2) + \ldots + 1 = n(n-1)/2 = O(n^2)$.

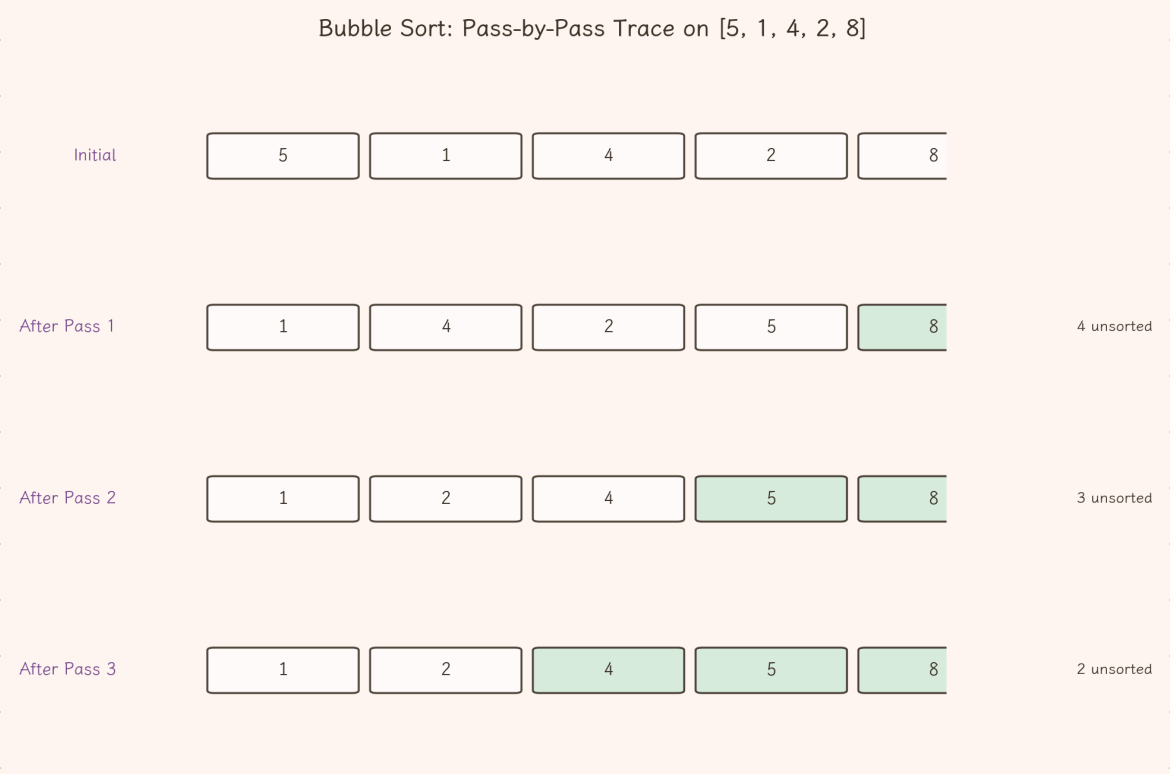


Figure 5: Bubble sort pass-by-pass trace on [5, 1, 4, 2, 8]

3.2 C++ Implementation

```cpp
void bubbleSort(vector<int>& a) {
  int n = a.size();
  for (int i = 0; i < n - 1; ++i) {
    bool swapped = false;
    for (int j = 0; j < n - 1 - i; ++j) {
      if (a[j] > a[j + 1]) {
        swap(a[j], a[j + 1]);
        swapped = true;
      }
    }
    if (!swapped) break; // already sorted
  }
}
```

Common Mistakes

> Inner loop range: must be **j < n-1-i** (not n-1) — otherwise recompares the already-sorted tail.
>
> Forgetting the early-stop flag wastes a full extra pass on sorted input.
>
> Using >= instead of > makes it unstable.

SECTION 4

Selection Sort

4.1 Basic Idea

Selection Sort finds the smallest element in the unsorted suffix via a linear scan, then swaps it into the next position of the sorted prefix. After pass i, A[1..i] holds the i smallest elements in their final order. The time complexity is $O(n^2)$ in all cases because the number of comparisons is always $(n-1) + (n-2) + \ldots + 1 = n(n-1)/2$, regardless of input order. The key advantage is that it performs exactly n-1 swaps (the minimum possible for any comparison sort), making it efficient when swap cost is high. However, it is **not stable** because the long-distance swap can leapfrog over equal keys.

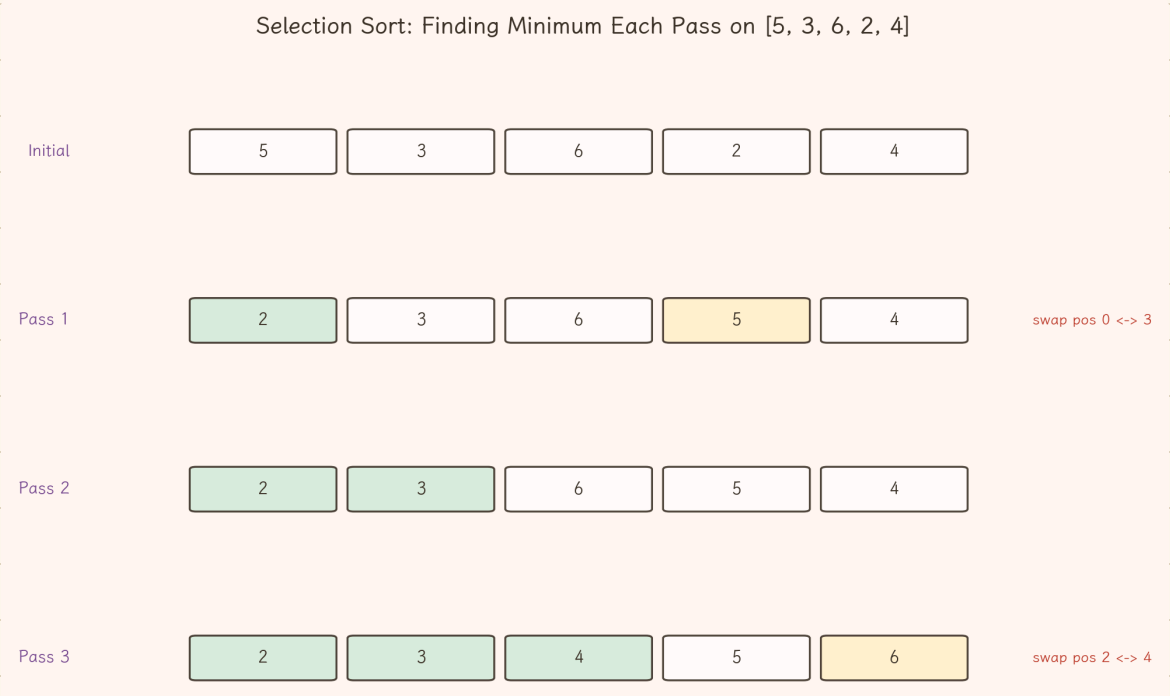


Figure 6: Selection sort minimum-finding passes on [5, 3, 6, 2, 4]

4.2 C++ Implementation

```cpp
void selectionSort(vector<int>& a) {
  int n = a.size();
  for (int i = 0; i < n - 1; ++i) {
    int mn = i;
    for (int j = i + 1; j < n; ++j)
      if (a[j] < a[mn]) mn = j;
    swap(a[i], a[mn]);
  }
}
```

Common Mistakes

- Swapping even when mn == i — harmless but wasteful; check first.
- Claiming stability — selection sort is **not stable** as written (long-distance swap).
- Comparisons still happen even when the array is sorted — $O(n^2)$ best case.

SECTION 5

Insertion Sort

5.1 Basic Idea

Insertion Sort builds a sorted prefix A[1..j-1] one element at a time by inserting A[j] into its correct position. It shifts larger elements one position to the right to create a gap, then places the key into that gap. This mimics how a human sorts playing cards in their hand. The algorithm is stable and in-place, and it is **adaptive**: its running time is proportional to the number of inversions in the input. For already-sorted input the best case is O(n); for reverse-sorted input the worst case is O(n²). It is efficient for small or nearly-sorted data and is used as a subroutine inside hybrid sorts like TimSort and Introsort.

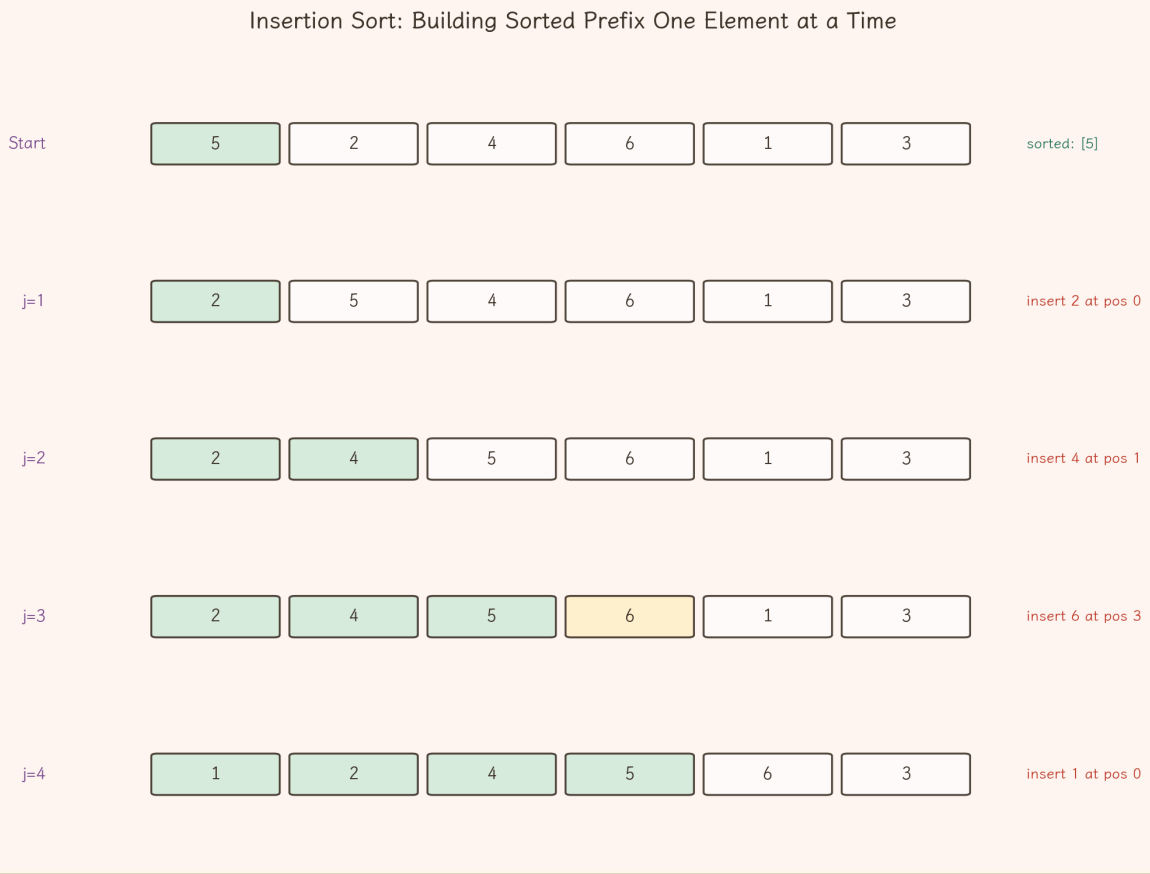


Figure 7: Insertion sort building the sorted prefix one element at a time

5.2 Loop Invariant (Proof of Correctness)

Invariant: At the start of each iteration of the outer for loop, the subarray A[1..j-1] consists of the elements originally in A[1..j-1] but in sorted order. **Initialization:** When j=2, A[1..1] has one element and is trivially sorted. **Maintenance:** The inner while loop shifts each A[i] > key one position right, then places key at A[i+1], making A[1..j] sorted. **Termination:** When j = n+1, A[1..n] (the entire array) is sorted.

5.3 C++ Implementation

```cpp
// 0-indexed version (A[0..n-1])
void insertionSort(vector<int>& a) {
  int n = a.size();
  for (int j = 1; j < n; ++j) {
    int key = a[j];
    int i = j - 1;
    while (i >= 0 && a[i] > key) {
      a[i + 1] = a[i]; // shift right
      --i;
    }
    a[i + 1] = key; // place key
  }
}
```

5.4 Complexity Analysis

Let t_j = number of while-loop tests for the current j. Then $T(n) = \Sigma\, t_j$ for j=2..n. In the worst case (reverse sorted), $t_j = j$, giving $T(n) = n(n+1)/2 - 1 = O(n^2)$. In the best case (already sorted), $t_j = 1$, giving $T(n) = n-1 = O(n)$. The average case (random input) gives $t_j \approx j/2$, so $T(n) = O(n^2)$. Space is O(1) auxiliary.

Common Mistakes

- Overwriting A[j] before saving it as key — always save first.
- Using >= in while loop makes the sort unstable.
- Off-by-one: i >= 0 (0-indexed) vs i > 0 (1-indexed Cormen style).
- Inserting at i instead of i+1 after the loop — wrong position.
- Forgetting best case is O(n), not $O(n^2)$ — examiners often ask this.

SECTION 6

Merge Sort

6.1 Basic Idea

Merge Sort is a divide-and-conquer algorithm that splits the array into two halves, recursively sorts each half, then merges the two sorted halves back together. All the work is in the merge step (combine); the divide step is just an index calculation. Merge Sort is stable but not in-place, requiring $\Theta(n)$ auxiliary space for the temporary arrays. It guarantees $\Theta(n \log n)$ time in all cases (best = average = worst), making it asymptotically optimal among comparison sorts. It is preferred for linked lists and external sorting due to its predictable, consistent performance.

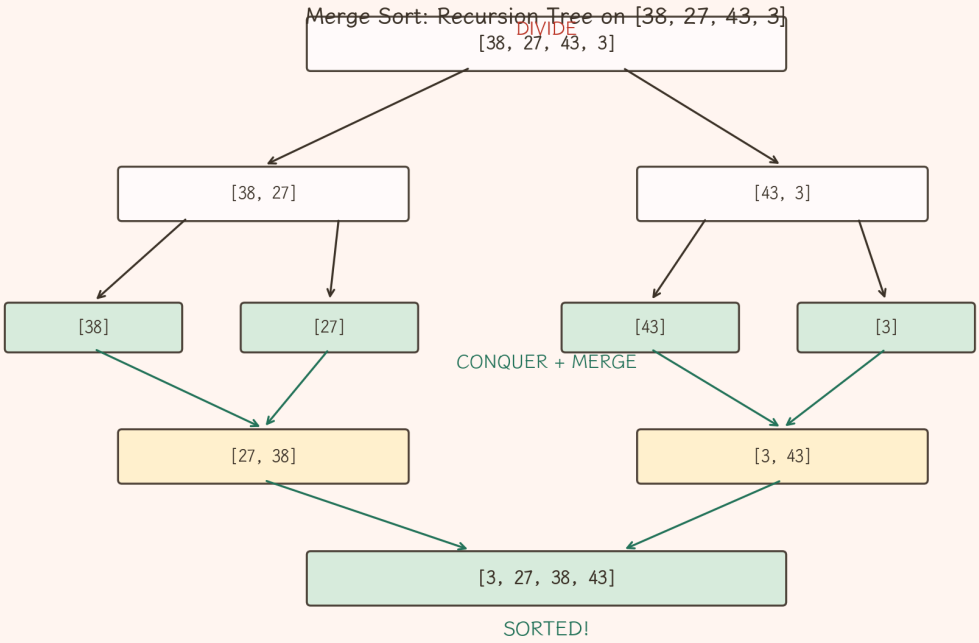


Figure 8: Merge sort recursion tree — divide, conquer, then merge

6.2 C++ Implementation

```
void merge(vector<int>& a, int p, int q, int r) {
  int n1 = q - p + 1, n2 = r - q;
  vector<int> L(a.begin() + p, a.begin() + q + 1);
  vector<int> R(a.begin() + q + 1, a.begin() + r + 1);
  int i = 0, j = 0, k = p;
  while (i < n1 && j < n2)
    a[k++] = (L[i] <= R[j]) ? L[i++] : R[j++]; // <= keeps stable
  while (i < n1) a[k++] = L[i++];
  while (j < n2) a[k++] = R[j++];
}

void mergeSort(vector<int>& a, int p, int r) {
  if (p >= r) return;
  int q = p + (r - p) / 2;
  mergeSort(a, p, q);
  mergeSort(a, q + 1, r);
  merge(a, p, q, r);
}
```

6.3 Recurrence & Complexity

$$T(n) = 2 \cdot T(n/2) + \Theta(n) \;\Rightarrow\; T(n) = \Theta(n \log n)$$

Substitution proof: $T(n) = 2T(n/2) + cn = 2^2T(n/4) + 2cn = \ldots = 2^kT(n/2^k) + k \cdot cn$. Setting $2^k = n$ gives $k = \lg n$, so $T(n) = n \cdot T(1) + cn \cdot \lg n = \Theta(n \log n)$. The recursion tree has depth $\lg n$ with work cn per level, totaling $\Theta(n \log n)$. Space is $\Theta(n)$ auxiliary for temp arrays plus $\Theta(\lg n)$ for the stack.

Bonus: Counting Inversions

Extend Merge to count inversions: when taking an element from the right subarray R[j], it is inverted with every remaining element in L (count += n1 - i). Complexity is the same as Merge Sort: Θ(n log n).

Common Mistakes

- Using < instead of ⩽ in merge — breaks stability.
- Wrong midpoint formula causes infinite recursion when r = p+1.
- Allocating temp arrays inside merge (O(n log n) allocations) — pass a single reusable buffer.

SECTION 7

Quick Sort

7.1 Basic Idea

Quick Sort is an in-place divide-and-conquer algorithm. It picks a pivot element, partitions the array so that elements ⩽ pivot are on its left and elements > pivot are on its right, then recurses on each side. The combine step is empty because all work happens during partitioning (unlike Merge Sort). Best and average case are Θ(n log n), but worst case degrades to Θ(n²) when the pivot consistently produces unbalanced splits, such as on already-sorted input with the first or last element as pivot. Despite this, Quick Sort is the fastest general-purpose sort in practice due to small constants and excellent cache behaviour. It is not stable but is in-place with O(lg n) stack space on average.

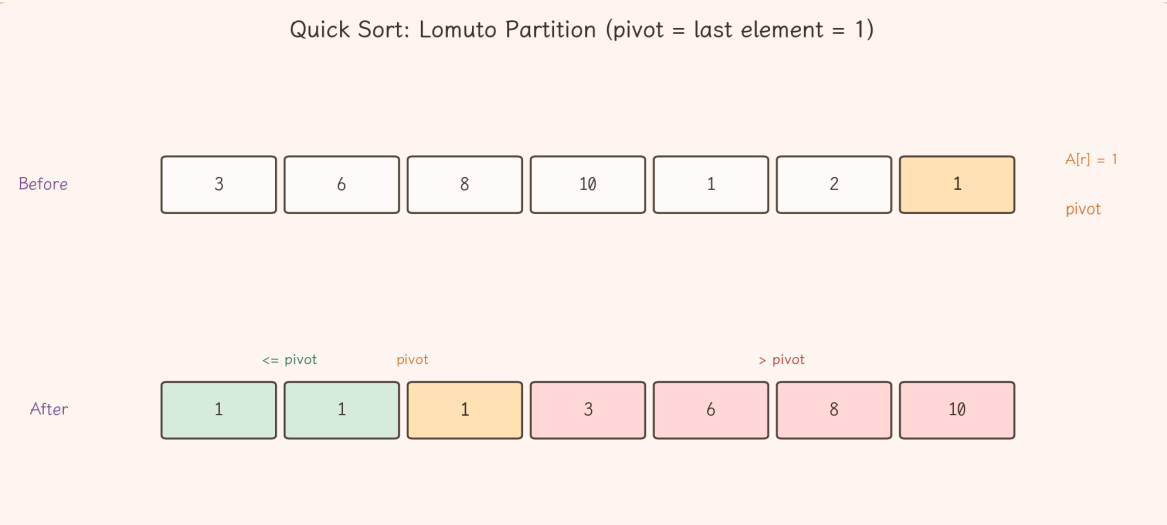


Figure 9: Lomuto partition — rearranging around the pivot element

7.2 Algorithm (Lomuto Partition)

Partition invariant: At any point during the loop, A[p..i] ⩽ pivot, A[i+1..j-1] > pivot, A[j..r-1] is unprocessed, and A[r] = pivot. After the loop, swap A[i+1] with A[r] to

place the pivot in its final sorted position, then return i+1 as the pivot index. The partition cost is Θ(n) since it performs a single linear scan.

7.3 C++ Implementation (Lomuto)

```
int partition(vector<int>& a, int p, int r) {
  int x = a[r]; // pivot = last element
  int i = p - 1;
  for (int j = p; j < r; ++j) {
    if (a[j] <= x) { ++i; swap(a[i], a[j]); }
  }
  swap(a[i + 1], a[r]);
  return i + 1;
}
void quickSort(vector<int>& a, int p, int r) {
  if (p < r) {
    int q = partition(a, p, r);
    quickSort(a, p, q - 1);
    quickSort(a, q + 1, r);
  }
}
```

7.4 Randomized Quick Sort

```
int randomizedPartition(vector<int>& a, int p, int r) {
  int i = p + rand() % (r - p + 1); // random index in [p, r]
  swap(a[i], a[r]); // move random pivot to end
  return partition(a, p, r); // then standard Lomuto
}
```

7.5 Complexity Analysis

Case	Partition Balance	Recurrence	Result
Best	Exact half (n/2, n/2)	T(n) = 2T(n/2) + Θ(n)	Θ(n log n)
Average	Random pivot	E[T(n)] = 2·E[T(n/2)] + Θ(n)	Θ(n log n)
Worst	Empty + (n-1) side	T(n) = T(n-1) + Θ(n)	Θ(n²)

7.6 Pivot Selection Strategies

Strategy	Cost	Notes
First / Last element	O(1)	Terrible on sorted input: O(n²)
Random element	O(1)	Eliminates worst-case input pattern; expected O(n log n)
Median of three	O(1)	Median of A[first], A[mid], A[last]. Robust in practice
True median	O(n)	Optimal but expensive; only theoretical guarantee

7.7 Merge Sort vs Quick Sort

Aspect	Merge Sort	Quick Sort	Winner
Worst-case time	Θ(n log n)	Θ(n²)	Merge
Average time	Θ(n log n)	Θ(n log n)	Tie (Quick has lower constants)

Auxiliary space	Θ(n)	O(lg n) stack	Quick
Stable	Yes	No	Merge
In-place	No	Yes	Quick
Cache friendly	No (random access)	Yes (sequential)	Quick
Linked lists	Yes	No (needs random access)	Merge

Common Mistakes

> Pivot = first/last on already-sorted input ⇒ guaranteed Θ(n²).
>
> Off-by-one in partition indices (i+1 vs i) — misplaces pivot.
>
> Forgetting base case if p < r ⇒ infinite recursion.
>
> Recursing on (p, q) and (q, r) instead of (p, q-1) and (q+1, r) ⇒ pivot re-enters, infinite loop.

SECTION 8

Counting Sort

8.1 Basic Idea

Counting Sort is a non-comparison sort that exploits knowledge of the value range k. It counts how many input elements have each value, then computes prefix sums to determine each element's final position. The time complexity is Θ(n + k) where k = max value, which is linear when k = O(n). The sort is stable because the placement step processes the input from right to left. It becomes impractical when k is much larger than n (memory and time blow up). The algorithm works in four clear steps: initialize the counter array, count occurrences, compute prefix sums, then place each element in its final position.

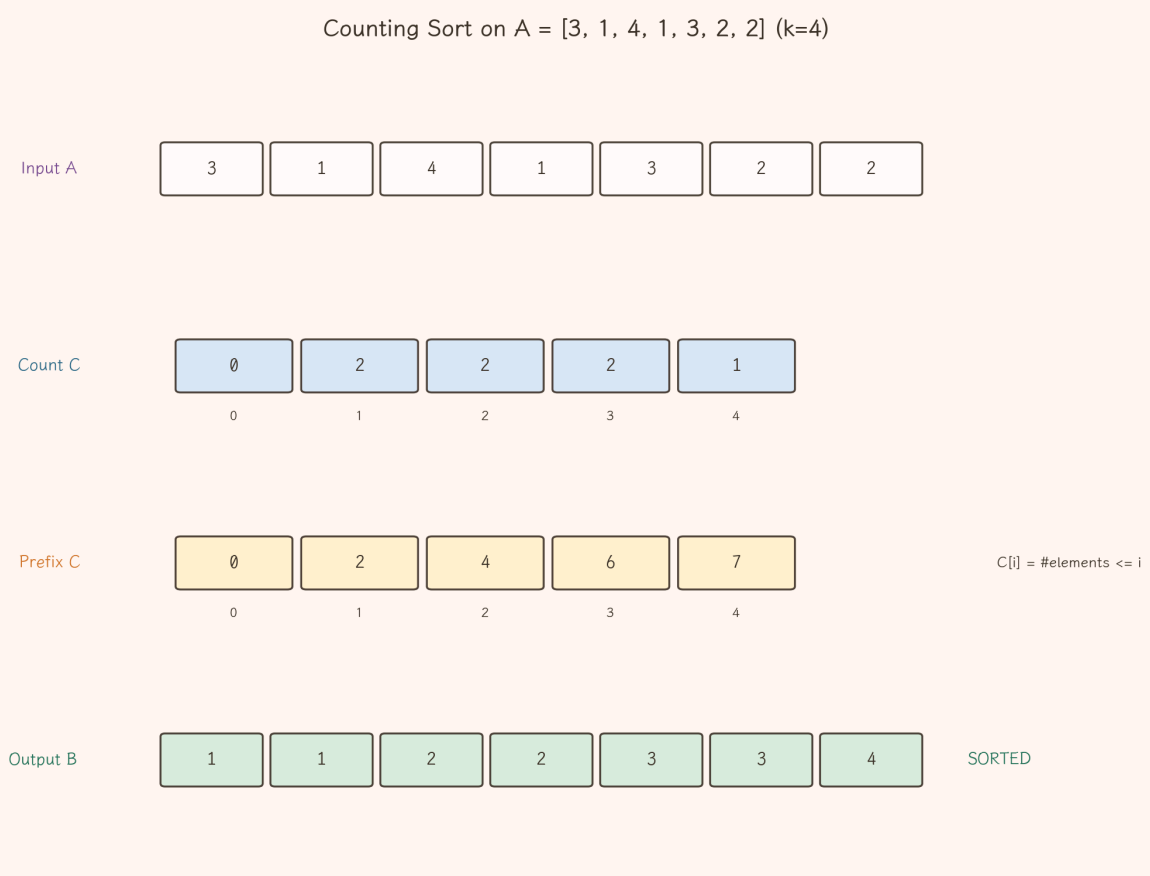


Figure 10: Counting sort — count, prefix sum, then place

8.2 C++ Implementation

```cpp
void countingSort(const vector<int>& a, vector<int>& b, int k) {
  int n = a.size();
  vector<int> C(k + 1, 0);
  for (int x : a) C[x]++;               // count
  for (int i = 1; i <= k; ++i) C[i] += C[i - 1]; // prefix sum
  b.resize(n);
  for (int j = n - 1; j >= 0; --j) {  // reverse => stable
    b[--C[a[j]]] = a[j];
  }
}
```

Common Mistakes

> Iterating placement step left-to-right — **breaks stability**. Must go right-to-left.

> Forgetting to decrement C[A[j]] after placement — all equal keys land in the same slot.

> Not handling negative keys — must offset by min value.

> Using C of size n instead of k+1 — out-of-bounds when max value > n.

SECTION 9

Radix Sort

9.1 Basic Idea

Radix Sort processes integers (or strings) one digit at a time, using a stable sort (usually Counting Sort) at each digit position. The two variants are LSD (Least Significant Digit first, more common) and MSD (Most Significant Digit first). The number of passes equals the number of digits d in the maximum key. Time complexity is O(d · (n + b)) where d = number of digits and b = base. The sort is stable, and this is mandatory because the stability of the inner sort is what makes Radix Sort correct — the order produced by earlier (less significant) digit passes must be preserved for equal keys in the current pass.

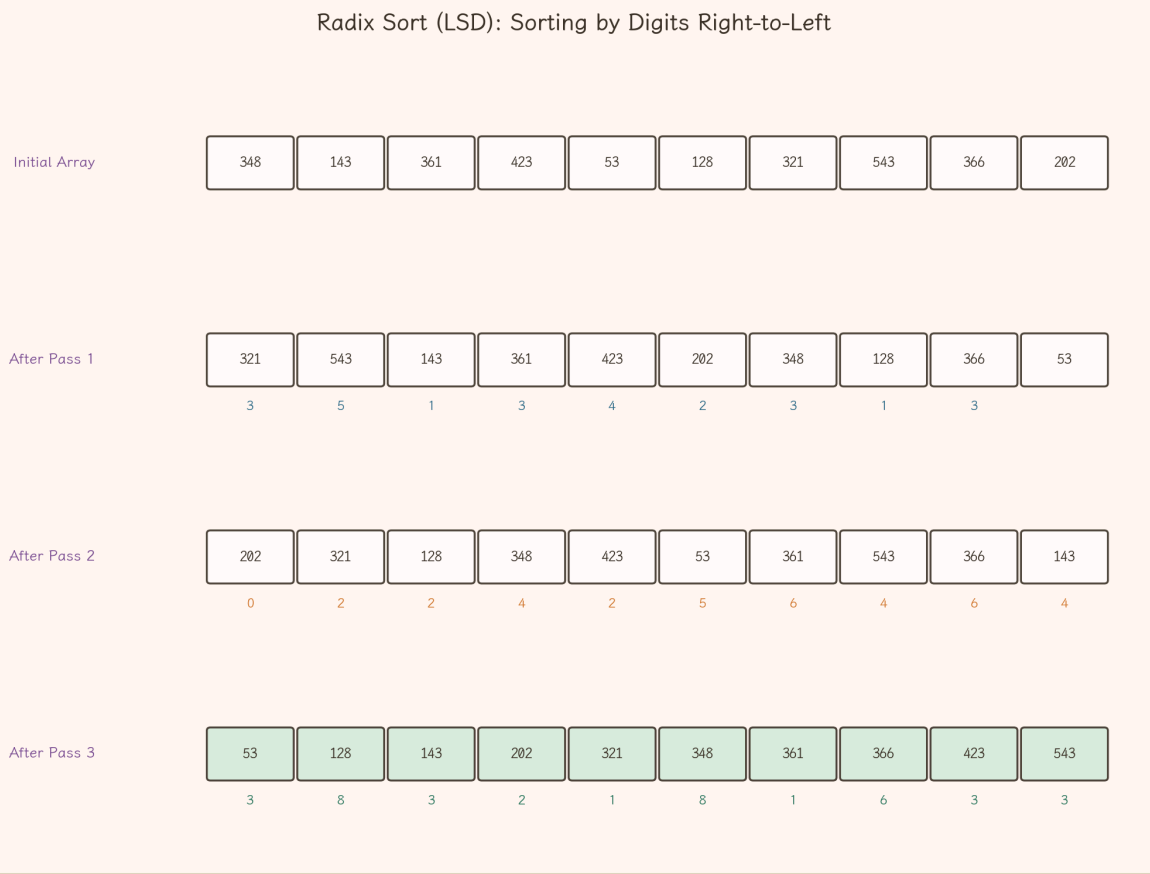


Figure 11: LSD Radix sort — sorting by units, tens, then hundreds digits

9.2 C++ Implementation (LSD, base 10)

```
void radixSort(vector<int>& a) {
  if (a.empty()) return;
  int mx = *max_element(a.begin(), a.end());
  for (int exp = 1; mx / exp > 0; exp *= 10)
    countingByDigit(a, exp);
}
void countingByDigit(vector<int>& a, int exp) {
  int n = a.size();
  vector<int> out(n), cnt(10, 0);
  for (int x : a) cnt[(x / exp) % 10]++;
```

```
  for (int i = 1; i < 10; ++i) cnt[i] += cnt[i - 1];
  for (int j = n - 1; j >= 0; --j) { // reverse => stable
    int d = (a[j] / exp) % 10;
    out[--cnt[d]] = a[j];
  }
  a.swap(out);
}
```

9.3 Complexity

Case	#digits d	Total Time
Best	$d = \log_b n$	O(n log n)
Worst	d = n	$O(n^2)$
Typical (fixed d)	d = constant	$O(d \cdot (n + b)) \approx O(n)$

Common Mistakes

> Inner sort **MUST be stable** — using an unstable sort breaks Radix Sort completely.
>
> Forgetting to handle leading zeros (pad shorter numbers).
>
> Base b too large: memory grows. Base b too small: more passes.
>
> Not handling negative numbers — split into negatives and positives, sort each, then concatenate.

SECTION 10

Comparison vs Non-Comparison Sorts

Aspect	Comparison Sorts	Non-Comparison Sorts	Notes
Lower bound	Ω(n log n)	None (can be O(n))	Counting, Radix, Bucket
Input assumption	Any comparable keys	Integer keys (known range)	Restricted domain
Examples	Merge, Quick, Bubble, Insertion, Selection	Counting, Radix, Bucket	
Stability	Varies	Usually stable	Radix requires stable inner sort
Adaptivity	Some (Insertion)	No	

SECTION 11

Formula & Complexity Quick Revision

11.1 Key Formulas to Memorize

Sum 1..n = n(n+1)/2 | lg(n!) = n·lg n - O(n) = Θ(n log n)

Master: T(n) = a·T(n/b) + f(n) | Binary Search: T(n) = T(n/2) + Θ(1) = Θ(lg n)

Merge/Quick best: T(n) = 2T(n/2) + Θ(n) = Θ(n log n) | Quick worst: T(n) = T(n-1) + Θ(n) = Θ(n²)

Counting: T(n) = Θ(n+k); linear iff k = O(n) | Radix: T(n) = O(d·(n+b)); needs stable inner sort

11.2 O(n²) Sorts Comparison

Aspect	Bubble	Selection	Insertion
Best time	O(n) [with flag]	O(n²)	O(n)
Worst time	O(n²)	O(n²)	O(n²)
Swaps (worst)	O(n²)	n-1 (minimum)	O(n²) shifts
Stable	Yes	No	Yes
Adaptive	Yes (with flag)	No	Yes (shifts = inversions)
Best for	Nearly sorted, teaching	Minimizing swaps	Small / nearly sorted

SECTION 12

Probable Exam Questions

Binary Search

E Write iterative Binary Search. Why is mid = lo + (hi-lo)/2 preferred?

M Prove that Binary Search runs in Θ(lg n) time using a recurrence.

H Given decision-tree lower bound, why can Binary Search achieve O(lg n) but comparison sorts cannot sort faster than Ω(n log n)?

Bubble / Selection Sort

E Trace Bubble Sort on {10, 20, 5, 100, 25, 6} for 3 passes.

E Why does Selection Sort perform at most n-1 swaps?

M Modify Bubble Sort to terminate early when the array is already sorted. Show best-case complexity.

M Compare the number of swaps in Bubble vs Selection Sort on a reverse-sorted array of size n.

Insertion Sort

E Dry-run INSERTION-SORT on {7, 4, 9, 5, 1} showing each pass.

E State the loop invariant of Insertion Sort and explain why it holds at termination.

E What is the best-case input for Insertion Sort? What is its complexity?

M Derive $T(n)$ for the worst case (reverse-sorted input) and show it is $O(n^2)$.

M Why is Insertion Sort preferred over Merge Sort for small subarrays ($n \leq 10$)?

H Show that running time of Insertion Sort is $O(n + I)$ where I is the number of inversions. Argue it is adaptive.

H Implement Insertion Sort on a singly linked list. Identify which steps differ from the array version.

Merge Sort

E Sort {5, 2, 4, 7, 1, 3, 2, 6} using Merge Sort; draw the recursion tree.

E State the recurrence for Merge Sort and its solution.

E Why is Merge Sort stable but Quick Sort not?

M Prove $T(n) = 2T(n/2) + cn = \Theta(n \log n)$ using the substitution method.

M Why does Merge Sort need $\Theta(n)$ extra space? Can it be made in-place?

M Modify Merge to count inversions. State the recurrence and final complexity.

H Why is Merge Sort preferred over Quick Sort for sorting linked lists? Give an in-place merge for linked lists.

Quick Sort

E Trace PARTITION on {2, 5, 8, 3, 9, 4, 1, 7, 10, 6} with pivot = 6 (last element).

E State the recurrence for Quick Sort in best and worst case.

E When does Quick Sort degrade to $O(n^2)$? How is this avoided?

M Implement Randomized-Partition. Why does randomization not change worst-case bound but improves expected bound?

M Trace Median-of-Three pivot selection on a sorted array of 10 elements.

M Compare Merge Sort and Quick Sort on (a) stability, (b) space, (c) worst-case time, (d) practical performance.

H Prove that the expected running time of Randomized Quick Sort is $O(n \log n)$.

H Modify Quick Sort so worst-case stack space is $O(\lg n)$ even on sorted input (tail-recursion elimination).

H Given an array of n identical elements, what is the running time of Lomuto-partition Quick Sort? Modify for duplicates (3-way partition / Dutch National Flag).

Counting / Radix Sort

E Trace Counting Sort on A = {3, 0, 3, 2, 0, 3, 5, 2} with k = 5. Show C after each step.

E Why must the placement loop in Counting Sort run from n downto 1?

E Sort {348, 143, 361, 423, 53, 128, 321, 543, 366, 202} using LSD Radix Sort; show the array after each pass.

M Prove that any comparison sort requires Ω(n log n) comparisons in the worst case.

M What is the time complexity of Counting Sort when k = O(n²)? When is it preferred over Merge Sort?

M Why must the inner sort in Radix Sort be stable? Show with a 2-digit example what breaks if it is not.

H Design a Radix Sort for n integers in [0, n³ - 1] that runs in O(n) time. Choose the base carefully and justify.

H How would you sort n integers in [1, n²] in O(n) time? Can this be done with a comparison sort? Why or why not?

Theory & Coding

E Define a comparison sort. Give two examples of non-comparison sorts.

E Define stability of a sorting algorithm. Why does it matter?

E Define in-place sorting. Which of Merge / Quick / Insertion are in-place?

M Explain the Divide & Conquer paradigm. Which sorting algorithms use it?

M Implement Merge Sort iteratively (bottom-up) without recursion.

M Implement Counting Sort for an array that may contain negative integers.

H Bug-fix: if (p <= r) { q = partition(a,p,r); qs(a,p,q); qs(a,q+1,r); } — identify and fix.

H Predict the output of Counting Sort if step 4 iterates from 1 to n (instead of n downto 1). Is it still sorted? Still stable?

H Modify Merge Sort to count inversions. Submit a complete C++ implementation and analyze its complexity.

SECTION 13

Memorize Before Entering the Hall

Key Definitions

Comparison sort: Sorts using only pairwise comparisons; Ω(n log n) lower bound.

Stable sort: Equal keys retain their original relative order.

In-place sort: Uses O(1) (or O(lg n)) auxiliary space.

Adaptive sort: Runs faster on nearly-sorted input (e.g., Insertion).

Divide & Conquer: Divide → Conquer (recurse) → Combine. Used by Binary Search, Merge Sort, Quick Sort.

Pivot: Element chosen to partition Quick Sort. Bad pivot ⇒ O(n²).

Loop Invariant: Property true at start of each loop iteration; used to prove correctness.

Common Pitfalls (Exam Favourites)

- Integer overflow in (lo+hi)/2 ⇒ use lo+(hi-lo)/2.
- Infinite recursion in Quick Sort: missing if p < r, or recursing on (p, q) instead of (p, q-1).
- Counting Sort placement left-to-right ⇒ unstable; must be right-to-left.
- Forgetting to decrement C[A[j]] after placement ⇒ all equal keys collide.
- Radix Sort with non-stable inner sort ⇒ wrong answer.
- Merge Sort forgetting to copy back from temp buffer ⇒ result lost.
- Insertion Sort using >= instead of > ⇒ unstable.
- Bubble Sort inner loop range: must be n-1-i, not n-1.
- Selection Sort: claiming it is stable — it is not as written.

Quick Recall Checklist

- Stable sorts: Bubble, Insertion, Merge, Counting, Radix — **not** Selection, **not** Quick.
- In-place sorts: Bubble, Selection, Insertion, Quick — **not** Merge, **not** Counting, **not** Radix.
- Quick Sort worst case: sorted/reverse-sorted input with first/last pivot.
- Counting Sort linear iff k = O(n); Radix Sort needs stable inner sort.
- Insertion Sort loop invariant: at start of each for iteration, A[1..j-1] is sorted.
- Merge Sort: Θ(n) aux space; Quick Sort: O(lg n) stack space.
- Decision-tree lower bound: n! leaves ⇒ height ≥ lg(n!) = Ω(n log n).